

# The Ultimate Java/Spring Boot Interview Handbook

## I. Index of Sections

1. Core Java (OOps, Logic, & Memory)
  2. Java 8+ (Streams, Lambda, & Collections)
  3. Spring Boot (Core, MVC, & Security)
  4. Data Persistence (Hibernate, JPA, & JDBC)
  5. Microservices Architecture
  6. Database Management (SQL & Theory)
  7. DevOps, Deployment, & Testing
  8. Coding, Logic, & Scenarios
- 

## 1. Core Java (OOps, Logic, & Memory)

1. **OOps Fundamentals:** Explain the 4 pillars (Abstraction, Encapsulation, Inheritance, Polymorphism).
2. **Diamond Problem:** What is the diamond problem in inheritance, and how does Java solve it using Interfaces?
3. **Inheritance Types:** What types of inheritance does Java support? Why is multiple inheritance not supported for classes?
4. **Keywords:** Difference between `final`, `finally`, and `finalize`.
5. **Volatile vs. Synchronized:** Explain the difference between the `volatile` keyword and `synchronized` blocks.
6. **Method Overloading vs. Overriding:** Provide examples. Can we change the access modifier/scope of a method when overriding?
7. **Parent-Child Access:** If Class A has private, protected, default, and public methods, which can be overridden in a different package?
8. **Memory Management:** What is the difference between Heap and Stack? What specific info is stored in each?
9. **PermGen vs. Metaspace:** Explain the difference and why Java 8 moved to Metaspace.
10. **Object Creation:** How many ways can you create an object? If a String is created with `new`, how many objects are created?
11. **Immutable Strings:** Why are Strings immutable? Explain the String Constant Pool.
12. **Abstract vs. Interface:** Why can we not create an object of an abstract class? Can we have non-abstract methods in an abstract class?
13. **Constructors:** Can we create a parameterized constructor in an abstract class? If yes, how do we set values?
14. **Garbage Collection:** Explain GC implementation and the algorithms used (e.g., G1GC).
15. **Wrapper Classes:** Why do we need wrapper classes? Compare Wrapper vs. Primitive.
16. **Generics:** What are Generics in Java, and why are they used?

17. **Exception Superclass:** What is the superclass of all Exceptions?
  18. **Exception Handling:** Explain `try-with-resources` vs. traditional `try-catch`. What are the methods `printStackTrace()` and `getMessage()`?
- 

## 2. Java 8+ (Streams, Lambda, & Collections)

1. **Java 8 Advantages:** Why did companies migrate from Java 7 to 8? List the key features.
  2. **Functional Interfaces:** How do you create one? What are the advantages? List common ones (Predicate, Function, Supplier, Consumer).
  3. **Functional vs. Marker Interface:** Explain the difference with examples.
  4. **Lambda Expressions:** How do they simplify code?
  5. **Optional Class:** How do you use `Optional`? Which inbuilt methods have you used to avoid null checks?
  6. **Streams API:** Explain `map()` vs. `flatMap()`. What are intermediate vs. terminal operations?
  7. **Stream Processing:** Explain Stream vs. Parallel Stream.
  8. **Collections Framework:** Explain the hierarchy. Difference between `ArrayList` and `LinkedList` (why is insertion faster in `LinkedList`?).
  9. **HashMap Working:** Explain internal hashing, the `put()` method, and collision handling.
  10. **HashMap variants:** `HashMap` vs. `ConcurrentHashMap` vs. `WeakHashMap`.
  11. **Fail-Fast vs. Fail-Safe:** Explain the difference in iterators.
  12. **HashSet vs. TreeSet:** Compare performance and ordering.
  13. **Sorting:** Difference between `Comparable` and `Comparator`.
- 

## 3. Spring Boot (Core, MVC, & Security)

1. **Annotations:** Explain `@SpringBootApplication` and its internal components.
2. **Layer Annotations:** Difference between `@Controller`, `@Service`, and `@Repository`. What happens if you swap them?
3. **Dependency Injection:** Constructor vs. Setter injection. Which is better?
4. **Bean Scopes:** Explain Singleton, Prototype, Request, and Session. Why is Singleton the default?
5. **Circular Reference:** What is it, and how do you solve it in Spring?
6. **Profiles:** How do you implement environment-specific configurations using `@Profile`?
7. **Auto-configuration:** What is it, and how can you disable specific auto-configurations?
8. **Spring MVC:** Which servlet acts as the front controller? Explain the request flow.
9. **Qualifiers:** When and how do you use the `@Qualifier` annotation?
10. **Async Processing:** How do you use `@Async` and `Callable` in Spring?
11. **Actuators:** What are they, and how do you push logs from actuators?
12. **Security:** How have you implemented security? Explain JWT workflow and SQL injection prevention.

13. **Error Handling:** How do you handle exceptions globally in Spring Boot?
  14. **Status Codes:** Scenarios for 404, 200, 401, 500. How do you solve a 401 Unauthorized error?
- 

## 4. Data Persistence (Hibernate & JPA)

1. **Repository Types:** CRUD Repository vs. JPA Repository. Can CRUD do pagination and sorting?
  2. **JPA Annotations:** `@PrePersist`, `@PreUpdate`, `@Transient`. Give use cases for `@Transient`.
  3. **Hibernate Joins:** How do you implement a Many-to-Many join? Explain the `@JoinTable`.
  4. **Caching:** Difference between L1 and L2 cache in Hibernate.
  5. **Session Management:** Difference between `Session` and `SessionFactory`.
  6. **Batching:** How do you achieve batch processing in Hibernate?
  7. **ID Generation:** What are the different strategies for ID generation in Hibernate?
- 

## 5. Microservices Architecture

1. **Monolithic vs. Microservices:** Key differences and when to choose which.
  2. **Communication:** How do two services communicate? (RestTemplate vs. OpenFeign).
  3. **Service Discovery:** Gateway vs. Discovery Server (Eureka).
  4. **Fault Tolerance:** How do you handle service failure and rollbacks?
  5. **Distributed Transactions:** How do you handle transactions across multiple services?
  6. **Design Patterns:** Which patterns have you used (e.g., Saga, Aggregator, Circuit Breaker)?
  7. **Externalization:** How do you handle configuration externalization?
- 

## 6. Database Management (SQL & Theory)

1. **SQL Commands:** Difference between `TRUNCATE`, `DROP`, and `DELETE`.
  2. **Joins:** Explain Inner, Left, Right, and Full Joins.
  3. **Key Constraints:** Primary Key vs. Unique Key vs. Foreign Key.
  4. **Normalization:** Explain the levels of normalization in a DB.
  5. **ACID Properties:** Explain each letter with respect to a transaction.
  6. **Procedures vs. Functions:** Key differences and use cases.
  7. **SQL Views:** What are they, and what DML operations can be performed on them?
- 

## 7. DevOps, Deployment, & Testing

1. **Docker:** How do you deploy using Docker? How do you push images to a registry?
2. **Kubernetes:** How do you handle deployment with K8s?
3. **Permissions:** How do you give folder/log permissions in a Docker container?
4. **Jenkins:** How do you write a Jenkins job for a Spring Boot project?
5. **Unit Testing:** How do you use JUnit to test methods with `if` conditions?
6. **Code Coverage:** How do you achieve coverage for static blocks and static methods? Have you used SonarQube?

---

## 8. Coding, Logic, & Scenarios

1. **Logic:** Print the first character of all strings in a list.
  2. **Sorting Logic:** Sort a list of students by Rank.
  3. **Algorithm:** Explain Bubble Sort and its manual implementation.
  4. **String Logic:** Check if a string is a Palindrome without using IDE functions.
  5. **Collection Logic:** Remove elements divisible by 5 from a list of integers (1-20). Why use an Iterator?
  6. **Real-world:** If you have an hourly temperature recording, how do you find the Min, Max, and Average for the day?
  7. **Output Question:** If `Atest atest = new BTest();`, which methods are called?
  8. **Query:** Write a query for the 2nd highest salary using `LIMIT` and `OFFSET`.
-